LAPORAN RESMI
MODUL III
PERULANGAN (LOOPING)
ALGORITMA DAN DASAR PEMROGRAMAN



NAMA	**: ADIT AGUSTIAN CANDRA**
N.R.P	**: 260404100126**
DOSEN	**: Dr. IMAMAH, S.Kom., M. Kom**
ASISTEN	**: ISMA' YAFA NUR ZAMZAMI RAMADHANI**
TGL PRAKTIKUM	**: 09 OKTOBER 2026**

Disetujui : 14 Oktober 2026
Asisten

Isma' Yafa Nur Zamzami Ramadhani
23.04.411.00081



LABORATORIUM BISNIS INTELIJEN SISTEM
PRODI SISTEM INFORMASI
JURUSAN TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS TRUNODJOYO MADURA

DAFTAR ISI

DAFTAR ISI ..... **i**
BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Tujuan 1
BAB II DASAR TEORI **2**
2.1 Pengertian Perulangan 2
2.2 Perintah for 2
2.3 Perintah While 3
2.4 Perintah Break, Continue dan Pass 3
2.4.1 Perintah Break 3
2.4.2 Perintah Continue 4
2.4.2 Statemen Perintah Pass 4
2.5 Praktikum 5
BAB III TUGAS PENDAHULUAN **7**
BAB IV IMPLEMENTASI **9**
4.1 Tugas Praktikum 9
4.1.1 Tugas Praktikum Soal No. 1 9
4.1.2 Tugas Praktikum Soal No. 2 9
4.1.3 Tugas Praktikum Soal No.3 9
4.1.4 Tugas Praktikum Soal No.4 10
4.2 Source Code 10
4.2.1 Source Code Soal No. 1 10
4.2.2 Source Code Soal No. 2 11
4.2.3 Source Code Soal No. 3 12
4.2.4 Source Code Soal No. 4 14
4.3 Hasil 15
4.3.1 Hasil Soal No. 1 15
4.3.2 Hasil Soal No. 2 15
4.3.3 Hasil Soal No. 3 15
4.3.4 Hasil Soal No. 4 15
4.3 Penjelasan 16

4.4.1 Penjelasan Soal No. 1 16
4.4.2 Penjelasan Soal No. 2 16
4.4.3 Penjelasan Soal No. 3 17
4.4.4 Penjelasan Soal No. 4 17
BAB V PENUTUP 19
5.1 Analisa 19
5.2 Kesimpulan 19

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pembuatan program, kita sering menemukan tugas yang harus dilakukan secara berulang. Contohnya adalah mencetak angka dari 1 hingga 100, menjumlahkan nilai banyak mahasiswa, atau menampilkan menu terus-menerus hingga pengguna memilih untuk keluar. Jika semuanya ditulis satu per satu, kode yang dihasilkan menjadi sangat panjang, melelahkan, dan rentan terhadap kesalahan. Bayangkan kita perlu menulis perintah print sebanyak seratus kali hanya untuk menunjukkan seratus angka. Di sinilah perulangan menjadi sangat penting.

Perulangan merupakan metode untuk membuat komputer melaksanakan serangkaian perintah secara berulang tanpa harus menulis perintah tersebut berkali-kali. Di dalam Python, terdapat dua perintah perulangan yang utama, yaitu for dan while. Perintah for biasanya digunakan ketika jumlah iterasi sudah ditentukan, sementara while digunakan ketika perulangan perlu terus berlangsung selama suatu kondisi masih benar. Selain itu, terdapat perintah break, continue, dan pass yang berfungsi untuk mengatur proses perulangan, seperti menghentikan perulangan di tengah jalan atau melewati satu iterasi tertentu.

Meskipun tampaknya mudah, perulangan juga memiliki hal-hal yang perlu diperhatikan. Jika kondisi dalam while tidak pernah menjadi salah, program akan terus berjalan tanpa henti, yang dikenal sebagai infinite loop. Oleh karena itu, memahami cara kerja perulangan dengan baik sangat penting, terutama bagi pemula yang baru mempelajari dunia pemrograman. Praktikum Modul III ini dirancang agar praktikan dapat memahami dan mencoba perulangan di Python, mulai dari for, while, hingga break, continue, dan pass.

1.2 Tujuan

- Mahasiswa mampu memahami penggunaan statement perulangan dan mampu membuat program dengan konsep perulangan

BAB II
DASAR TEORI

2.1 Pengertian Perulangan

Perintah perulangan digunakan untuk mengulang pengeksekusian statemen-statemen hingga berkali-kali sesuai dengan iterasi yang diinginkan. Dalam python, perintah untuk perulangan (*loop*) adalah *for* dan *while*.

2.2 Perintah for

Perintah *for* dalam python mempunyai ciri khas tersendiri dibandingkan dengan bahasa pemrograman lain. Tidak hanya mengulang bilangan-bilangan sebuah ekspresi aritmatik, atau memberikan keleluasaan dalam mendefinisikan iterasi perulangan dan menghentikan perulangan pada saat kondisi tertentu. Dalam python, statemen *for* bekerja mengulang berbagai macam tipe data sekuensial seperti List, String, dan Tuple.

Bentuk umum perintah *for*,

```
for (variabel) in (objek) :
    statemen
else:
    statemen
```

Contoh 1:

```
for i in range(1, 10, 2):
    print(i)
```

Pada contoh 1, perintah perulangan terjadi dimana digunakan untuk menampilkan data pada variabel i berdasarkan nilai yang ditentukan pada fungsi range() dan terdapat 3 parameter yaitu 1 sebagai nilai awal, 10 sebagai nilai akhir dan 2 sebagai nilai kenaikan.

Contoh 2:

```
for i in range(1, 6):
    print("Perulangan ke-", i)
```

Pada contoh 2, perulangan *for* digunakan untuk menjalankan pengulangan dari angka 1 hingga 5 dengan kenaikan 1 sebagai default.

2.3 Perintah While

Perintah *while* pada python merupakan perintah yang paling umum digunakan untuk proses iterasi. Konsep sederhana dari perintah *while* adalah ia akan mengulang mengeksekusi statemen dalam blok *while* selama nilai kondisinya benar. Dan ia akan keluar atau tidak melakukan eksekusi blok statemen jika nilai kondisinya salah.

Bentuk umum statemen *while*,

```
while (kondisi) :
   statemen
```

Pada contoh pertama, merupakan contoh sederhana penggunaan *while*. Pada contoh di atas, program akan terus mengeksekusi statemen dalam badan *while*, dikarenakan kondisinya selalu benar (*true*). Kondisi seperti ini disebut *infinite loop*.

Contoh 1:

```
x = "wulan"

while x:
   print(x)
   x = x[1:]
```

Contoh 2:

```
a = 0
b = 10

while a < b:
   print(a)
   a = a + 1
```

2.4 Perintah Break, Continue dan Pass

2.4.1 Perintah Break

Perintah *break* digunakan untuk menghentikan jalannya proses iterasi pada statemen *for* atau *while*. Statemen yang berada di bawah break tidak akan di eksekusi dan program akan keluar dari proses looping.

Contoh *break*:

```
x = 1


while x < 5:
    if x == 3:
        break
    print(x)
    x = x + 1
else:
    print("Selesai")
```

2.4.2 Perintah Continue

Statemen *continue* menyebabkan alur program kembali ke perintah *looping*. Jadi jika dalam sebuah perulangan terdapat statemen *continue*, maka program akan kembali ke perintah *looping* untuk iterasi selanjutnya.

Contoh *continue*:

```
n = 10
while n:
    n = n - 1
    if n % 2 != 0:
        continue
    print(n)
```

2.4.2 Statemen Perintah Pass

pass mengakibatkan program tidak melakukan tindakan apa-apa. Perintah *pass* biasanya digunakan untuk mengabaikan suatu blok statemen perulaang.

```
for i in range(1, 6):
    if i == 3:
        pass
    else:
        print(i)
```

Pada contoh di atas, pass digunakan sebagai perintah kosong, yaitu ketika kondisi i == 3 terpenuhi, Python tidak melakukan tindakan apa pun dan langsung melakukan perulangan berikutnya.

2.5 Praktikum

Contoh 1:

```
a = 1
while a < 5:
   print(a)
   a+=1
```

Contoh 2 menampilkan bilangan ganjil:

```
a = 0
while a < 10:
   a+=1
   if a% 2:
      print("%d bilangan ganjil")
   else:
      continue
```

Contoh 3 menampilkan angka 1 sampai 6dengan menggunakan while:

```
a = 1
while a<10:
   print(a)
   a+=1
   if a>6:
      break
```

Contoh 4 perintah while tanpa henti:

```
while 1:
   print('perulangan tiada batas, tekan ctrl + c untuk berhenti')
```

Contoh 5 menampilkan bilangan genap:

```
for a in range(1, 11):
   if a % 2 == 0:
      print(a)
```

Contoh 6 melewati angka 5 dengan continue:

```
for a in range(1, 11):
    if a == 5:
        continue
    print(a)
```

BAB III
TUGAS PENDAHULUAN

3.1 Soal

1. Jelaskan konsep Perulangan (For dan while) dalam bahasa Pemrograman Python menggunakan bahasamu sendiri! Apa peran utama Perulangan dalam efisiensi Penulisan kode Program?
2. Buatlah sebuah Program menggunakan Perulangan For di Python untuk mencetak semua bilangan genap mulai dari angka 2 sampai 20 dan jelaskan Program tersebut!
3. Apa yang dimaksud dengan kondisi Infinite Loop Pada Perulangan while?
   - Jelaskan Faktor utama yang menyebabkannya terjadi!
   - Bagaimana cara terbaik untuk mencegah terjadinya infinite loop dalam Program Python?
4. Jelaskan Fungsi dari Perulangan break, continue, dan Pass!
5. Seorang Dosen ingin merekap total nilai dari beberapa tugas mahasiswa yang terkumpul. Program akan meminta input nilai tugas satu per satu. Untuk mengakhiri Penginputan, Dosen cukup memasukkan angka -99.
   Setelah selesai, Program harus menampilkan :
   - Total akumulasi nilai seluruh tugas.
   - Banyaknya tugas yang sudah diinputkan.

3.2 Jawaban

1. Perulangan adalah cara menyuruh Program mengerjakan hal yang sama berkali-kali tanpa tuiis ulang kodenya. Ada For dipakai jika jumlah pengulangannya sudah jelas dan while dipakai jika jumlah pengulangannya belum pasti. Program terus berjalan dan baru berhenti ketika kondisinya salah. Perannya penting untuk kode menjadi singkat dan rapi
2. For i in range (2,21,2):
       Print (i)

   Penjelasan:
   - range (2, 21, 2) dimulai dari 2, berhenti sebelum 21 sebagai angka batas akhir karena angka batas akhir tidak dihitung, dan 2 diakhir sebagai kelipatan
   - For dipakai untuk Perulangan
3. Infinite loop adalah Perulangan yang tidak pernah berhenti karena kondisinya selalu benar. Penyebab utamanya biasanya karena variabel yang dicek tidak pernah berubah atau karena kondisinya dibuat selalu benar tanpa ada break. Cara untuk mencegahnya adalah memastikan ada bagian di dalam Perulangan yang mengubah variabel kondisi dan break yang jelas untuk keluar.

4 a. break : menghentikan seluruh perulangan

b. continue : melewati sisa kode di putaran yang sedang berjalan

c. Pass : tidak melakukan apa-apa

5. total = 0
   jumlah = 0

```
while True:
    nilai = int(input("masukkan nilai tugas : "))
    IF nilai == -gg:
        break
    total = total + nilai
    jumlah = jumlah + 1
Print ("Total nilai seluruh tugas:", total)
Print ("Banyaknya tugas yang diinput:", jumlah)
```

Program ini meminta dosen memasukkan nilai tugas terus-menerus lalu menjumlahkannya total menyimpan jumlah semua nilai dan jumlah menghitung ada berapa tugas. while True membuat program terus meminta nilai tanpa henti sampai ada perintah untuk berhenti. di program ini -gg adalah sebagai perintah berhenti dengan break.

BAB IV
IMPLEMENTASI

4.1 Tugas Praktikum

4.1.1 Tugas Praktikum Soal No. 1

Pak Budi adalah kasir di Terminal Bus Sidoarjo. Tarif dihitung dari jarak tempuh
penumpang:

a. Jarak ≤ 5 km: Rp3.000
b. Jarak 6 sampai 15 km: Rp3.000 + Rp1.000 untuk setiap km di atas 5
c. Jarak > 15 km: Rp13.000 + Rp500 untuk setiap km di atas 15

Setiap penumpang ke-5, ke-10, dan seterusnya mendapat diskon 50%. Kasir terus memasukkan jarak penumpang sampai memasukkan 0. Jarak negatif dianggap tidak valid dan harus diulang tanpa dihitung sebagai penumpang. Di akhir, tampilkan jumlah penumpang, total pendapatan, jarak terjauh, dan rata-rata jarak.

4.1.2 Tugas Praktikum Soal No. 2

Arsitek kota merancang Menara Lampu Hias berbentuk belah ketupat dari angka. Program meminta tinggi setengah menara n (valid jika 2 sampai 15, ulangi jika di luar rentang). Setiap baris berisi angka naik dari 1 sampai nomor baris, lalu turun kembali ke 1. Angka yang lebih dari satu digit ditampilkan sebagai angka terakhirnya saja (misalnya 10 menjadi 0). Bagian atas membesar sampai baris ke-n, kemudian mengecil kembali, dan setiap baris rata tengah.

4.1.3 Tugas Praktikum Soal No.3

Di sebuah universitas, seorang dosen ingin membuat program sederhana untuk mengelola nilai mahasiswa dalam satu kelas yang terdiri dari 10 mahasiswa. Setiap mahasiswa memiliki nilai tugas, nilai UTS, dan nilai UAS yang harus dimasukkan melalui program. Nilai akhir mahasiswa dihitung dengan bobot nilai tugas sebesar 30%, UTS sebesar 30%, dan UAS sebesar 40%. Setelah nilai akhir diperoleh, tentukan kategori nilai mahasiswa dengan ketentuan nilai 85–100 mendapatkan A, 75–84 mendapatkan B, 65–74 mendapatkan C, 50–64 mendapatkan D, dan nilai di bawah 50 mendapatkan E. Buatlah sebuah program yang dapat memproses seluruh data mahasiswa tersebut dan menampilkan nama, nilai tugas, nilai UTS, nilai UAS, nilai akhir, serta kategori nilai masing-masing

mahasiswa. Setelah semua data selesai diproses, program juga harus menampilkan jumlah mahasiswa yang memperoleh masing-masing kategori nilai, nilai akhir tertinggi beserta nama mahasiswanya, nilai akhir terendah beserta nama mahasiswanya, serta rata-rata nilai akhir seluruh mahasiswa.

4.1.4 Tugas Praktikum Soal No.4

Sebuah universitas ingin membuat program untuk melakukan seleksi awal penerima beasiswa. Program akan memproses data 10 mahasiswa secara bergantian. Setiap mahasiswa harus memasukkan nama, IPK, dan penghasilan orang tua per bulan. Mahasiswa dengan IPK kurang dari 3,00 dianggap tidak memenuhi syarat dan datanya tidak perlu diproses lebih lanjut, sehingga program langsung melanjutkan ke mahasiswa berikutnya. Jika IPK memenuhi syarat, program akan memeriksa penghasilan orang tua. Mahasiswa dengan penghasilan orang tua kurang dari atau sama dengan Rp3.000.000 mendapatkan beasiswa penuh, sedangkan penghasilan lebih dariRp3.000.000 sampai Rp5.000.000 mendapatkan beasiswa sebagian. Jika penghasilan lebih dari Rp5.000.000, mahasiswa tidak mendapatkan beasiswa. Program juga memiliki batas jumlah mahasiswa yang dapat memperoleh beasiswa penuh, yaitu 3 mahasiswa. Jika sudah terdapat 3 mahasiswa yang mendapatkan beasiswa penuh, proses seleksi dapat dihentikan dan program tidak perlu memproses mahasiswa berikutnya. Buatlah program Python yang dapat menampilkan hasil seleksi setiap mahasiswa serta menggunakan mekanisme untuk melewati mahasiswa yang tidak memenuhi syarat dan menghentikan proses ketika kuota beasiswa penuh telah terpenuhi.

4.2 Source Code

4.2.1 Source Code Soal No. 1

```
jumlah = 0
total_pendapatan = 0
total_jarak = 0
jarak_terjauh = 0

jarak = int(input("Masukkan jarak : "))

while jarak != 0:
    if jarak < 0:
```

```
        print("Jarak negatif tidak valid")
    else:
        jumlah = + 1
        total_jarak = total_jarak + jarak

        if jarak > jarak_terjauh:
            jarak_terjauh = jarak

        if jarak <= 5:
            tarif = 3000
        elif jarak <= 15:
            tarif = 3000 + (jarak - 5) * 1000
        else:
            tarif = 13000 + (jarak - 15) * 500

        if jumlah % 5 == 0:
            tarif = tarif / 2
            print("Penumpang ke-", jumlah, "dapat diskon 50%")

        print("Tarif: Rp", tarif)
        total_pendapatan = total_pendapatan + tarif

    jarak = int(input("Masukkan jarak:"))

if jumlah > 0:
    rata_rata = total_jarak / jumlah
else:
    rata_rata = 0

print("Jumlah penumpang :", jumlah)
print("Total pendapatan : Rp", total_pendapatan)
print("Jarak terjauh    :", jarak_terjauh, "km")
print("Rata-rata jarak  :", rata_rata, "km")
```

4.2.2 Source Code Soal No. 2

```
n = int(input("Masukkan tinggi (2-15): "))
```

```
while n < 2 or n > 15:
    print("Tinggi harus 2 sampai 15!")
    n = int(input("Masukkan tinggi (2-15): "))

for i in range(1, 2 * n):
    if i <= n:
        baris = i
    else:
        baris = 2 * n - i

    print(" " * (n - baris), end="")

    for j in range(1, baris + 1):
        print(j % 10, end="")

    for j in range(baris - 1, 0, -1):
        print(j % 10, end="")

    print()
```

4.2.3 Source Code Soal No. 3

```
jumlah_A = 0
jumlah_B = 0
jumlah_C = 0
jumlah_D = 0
jumlah_E = 0

total = 0
tertinggi = -1
terendah = 101

for i in range(1, 11):
    print("Mahasiswa ke -", i)

    nama = input("Nama: ")
    tugas = int(input("Nilai tugas: "))
    uts = int(input("Nilai UTS: "))
```

```
    uas = int(input("Nilai UAS: "))

    akhir = tugas * 0.3 + uts * 0.3 + uas * 0.4

    if akhir >= 85:
        kategori = "A"
        jumlah_A = jumlah_A + 1
    elif akhir >= 75:
        kategori = "B"
        jumlah_B = jumlah_B + 1
    elif akhir >= 65:
        kategori = "C"
        jumlah_C = jumlah_C + 1
    elif akhir >= 50:
        kategori = "D"
        jumlah_D = jumlah_D + 1
    else:
        kategori = "E"
        jumlah_E = jumlah_E + 1

    print("Nama:", nama)
    print("Nilai akhir:", akhir)
    print("Kategori:", kategori)

    total = total + akhir

    if akhir > tertinggi:
        tertinggi = akhir
        nama_tertinggi = nama

    if akhir < terendah:
        terendah = akhir
        nama_terendah = nama

print("Jumlah kategori A:", jumlah_A)
print("Jumlah kategori B:", jumlah_B)
```

```
print("Jumlah kategori C:", jumlah_C)
print("Jumlah kategori D:", jumlah_D)
print("Jumlah kategori E:", jumlah_E)
print("Nilai tertinggi:", tertinggi)
print("Nama nilai tertinggi:", nama_tertinggi)
print("Nilai terendah:", terendah)
print("Nama nilai terendah:", nama_terendah)
print("Rata-rata kelas:", total / 10)
```

4.2.4 Source Code Soal No. 4

```
jumlah_penuh = 0

for i in range(1, 11):
    print("Mahasiswa ke-", i)
    nama = input("Nama: ")
    ipk = float(input("IPK: "))

    if ipk < 3.00:
        print(nama, "tidak memenuhi syarat.")
        continue

    penghasilan = int(input("Penghasilan orang tua per bulan: "))

    if penghasilan <= 3000000:
        print(nama, "mendapat beasiswa penuh.")
        jumlah_penuh = jumlah_penuh + 1
        if jumlah_penuh == 3:
            print("Kuota beasiswa terpenuhi, seleksi dihentikan.")
            break
    elif penghasilan <= 5000000:
        print(nama, "mendapat beasiswa sebagian.")
    else:
        print(nama, "tidak mendapat beasiswa.")

print("Jumlah penerima beasiswa penuh:", jumlah_penuh)
```

4.3 Hasil

4.3.1 Hasil Soal No. 1

```
Masukkan jarak : 5
Tarif: Rp 3000
Masukkan jarak:7
Tarif: Rp 5000
Masukkan jarak:15
Tarif: Rp 13000
Masukkan jarak:16
Tarif: Rp 13500
Masukkan jarak:2
Penumpang ke- 5 dapat diskon 50%
Tarif: Rp 1500.0
Masukkan jarak:0
Jumlah penumpang : 5
Total pendapatan : Rp 36000.0
Jarak terjauh    : 16 km
Rata-rata jarak  : 9.0 km
```

4.3.2 Hasil Soal No. 2

```
Masukkan tinggi (2-15): 3
  1
 121
12321
 121
  1
```

4.3.3 Hasil Soal No. 3

```
Nilai tertinggi: 99.0
Nama nilai tertinggi: mhs8
Nilai terendah: 79.0
Nama nilai terendah: mhs7
Rata-rata kelas: 86.59
```

4.3.4 Hasil Soal No. 4

```
Mahasiswa ke- 1
Nama: mhs1
IPK: 3.10
Penghasilan orang tua per bulan: 3000000
mhs1 mendapat beasiswa penuh.
Mahasiswa ke- 2
Nama: mhs2
IPK: 3.5
Penghasilan orang tua per bulan: 2500000
mhs2 mendapat beasiswa penuh.
Mahasiswa ke- 3
Nama: mhs3
IPK: 3.80
Penghasilan orang tua per bulan: 3500000
mhs3 mendapat beasiswa sebagian.
Mahasiswa ke- 4
Nama: mhs4
IPK: 3.6
Penghasilan orang tua per bulan: 3200000
mhs4 mendapat beasiswa sebagian.
Mahasiswa ke- 5
Nama: mhs6
IPK: 3.5
Penghasilan orang tua per bulan: 250000
mhs6 mendapat beasiswa penuh.
Kuota beasiswa terpenuhi, seleksi dihentikan.
Jumlah penerima beasiswa penuh: 3
```

4.3 Penjelasan

4.4.1 Penjelasan Soal No. 1

Program ini dimulai dengan membuat empat variabel yang awalnya bernilai 0, yaitu jumlah, total_pendapatan, total_jarak, dan jarak_terjauh. Setelah itu kasir diminta memasukkan jarak penumpang pertama. Perulangan while jarak != 0 dipakai supaya program terus berjalan selama jarak yang dimasukkan bukan 0, jadi kalau kasir mengetik 0, perulangan langsung berhenti. Di dalam perulangan, program cek dulu jaraknya. Kalau jarak < 0, program menampilkan "Jarak negatif tidak valid" dan penumpang itu tidak dihitung. Kalau jarak valid, jumlah penumpang ditambah 1, jarak dimasukkan ke total_jarak, dan kalau jaraknya lebih besar dari jarak_terjauh maka jarak_terjauh diganti dengan jarak itu. Tarif dihitung dengan if-elif-else: jarak <= 5 tarifnya 3000, jarak <= 15 tarifnya 3000 + (jarak - 5) * 1000, dan sisanya (di atas 15 km) tarifnya 13000 + (jarak - 15) * 500. Lalu ada pengecekan jumlah % 5 == 0. Artinya kalau jumlah penumpang habis dibagi 5 (penumpang ke-5, ke-10, dan seterusnya), tarif dibagi 2 sebagai diskon 50% dan program menampilkan info diskon. Tarif kemudian ditampilkan dan ditambahkan ke total_pendapatan. Di akhir setiap putaran, program meminta jarak berikutnya. Setelah perulangan selesai, rata-rata jarak dihitung dari total_jarak dibagi jumlah. Ada pengecekan if jumlah > 0 supaya tidak terjadi pembagian dengan nol kalau belum ada penumpang sama sekali. Terakhir, program menampilkan jumlah penumpang, total pendapatan, jarak terjauh, dan rata-rata jarak.

4.4.2 Penjelasan Soal No. 2

Program meminta tinggi menara n. Perulangan while n < 2 or n > 15 membuat program terus meminta input ulang sampai n berada di antara 2 sampai 15. Kalau salah, muncul pesan "Tinggi harus 2 sampai 15!". Setelah n valid, perulangan for i in range(1, 2 * n) dipakai untuk membuat semua barisnya. Jumlah barisnya ada 2n - 1, yaitu bagian atas yang membesar sampai baris ke-n lalu bagian bawah yang mengecil. Variabel baris menentukan isi tiap baris: kalau i <= n maka baris = i (membesar), kalau tidak maka baris = 2 * n - i (mengecil). Untuk mencetak satu baris, pertama program mencetak spasi sebanyak n - baris supaya posisinya rata tengah, memakai end="" supaya tidak pindah baris. Kemudian for j in range(1, baris + 1) mencetak angka naik dari 1 sampai nomor baris, dan for j in range(baris

- 1, 0, -1) mencetak angka turun kembali ke 1. Angka dicetak dengan j % 10 sehingga angka dua digit hanya diambil digit terakhirnya (misalnya 10 jadi 0). Terakhir print() dipakai untuk pindah ke baris berikutnya.

4.4.3 Penjelasan Soal No. 3

Program ini membuat variabel jumlah_A sampai jumlah_E untuk menghitung banyaknya tiap kategori, total untuk menjumlahkan nilai akhir, serta tertinggi = -1 dan terendah = 101 sebagai nilai awal. Nilai awal ini sengaja dipilih supaya nilai mahasiswa pertama pasti menggantikannya. Perulangan for i in range(1, 11) dipakai karena jumlah mahasiswanya sudah pasti, yaitu 10 orang. Pada setiap putaran, program meminta nama, nilai tugas, nilai UTS, dan nilai UAS. Nilai akhir dihitung dengan rumus akhir = tugas * 0.3 + uts * 0.3 + uas * 0.4. Lalu if-elif-else menentukan kategorinya: >= 85 dapat A, >= 75 dapat B, >= 65 dapat C, >= 50 dapat D, dan sisanya E. Setiap kategori yang cocok langsung menambah 1 ke variabel jumlahnya. Setelah itu program menampilkan nama, nilai akhir, dan kategori mahasiswa tersebut. Masih di dalam perulangan, nilai akhir ditambahkan ke total. Kalau akhir lebih besar dari tertinggi, maka nilai dan nama mahasiswa disimpan sebagai yang tertinggi. Begitu juga kalau akhir lebih kecil dari terendah, disimpan sebagai yang terendah. Setelah 10 mahasiswa selesai, program menampilkan jumlah tiap kategori, nilai tertinggi beserta namanya, nilai terendah beserta namanya, dan rata-rata kelas yang didapat dari total dibagi 10.

4.4.4 Penjelasan Soal No. 4

Program ini memakai variabel jumlah_penuh yang awalnya 0 untuk menghitung berapa mahasiswa yang sudah dapat beasiswa penuh. Perulangan for i in range(1, 11) dipakai untuk memproses 10 mahasiswa satu per satu. Di setiap putaran, program meminta nama dan IPK. Kalau IPK < 3.00, program menampilkan bahwa mahasiswa itu tidak memenuhi syarat, lalu memakai continue. Artinya sisa kode di putaran itu dilewati (penghasilan tidak ditanyakan) dan program langsung lanjut ke mahasiswa berikutnya. Kalau IPK memenuhi syarat, baru program meminta penghasilan orang tua dan mengeceknya dengan if-elif-else: penghasilan <= 3000000 dapat beasiswa penuh, penghasilan <= 5000000 dapat beasiswa sebagian, dan di atas itu tidak mendapat beasiswa. Setiap kali ada yang mendapat beasiswa penuh, jumlah_penuh ditambah 1. Kalau jumlah_penuh sudah sama

dengan 3, program menampilkan pesan bahwa kuota terpenuhi lalu memakai break untuk menghentikan seluruh perulangan, jadi mahasiswa sisanya tidak diproses lagi. Setelah perulangan selesai, program menampilkan jumlah penerima beasiswa penuh

BAB V
PENUTUP

5.1 Analisa

Dari hasil praktikum, praktikan menganalisa bahwa perulangan sangat membantu karena perintah yang sama tidak perlu ditulis berkali-kali. Perulangan for dipakai jika jumlah pengulangannya sudah jelas, seperti pada soal nomor 2, 3, dan 4 yang memproses data dalam jumlah tetap. Pada soal nomor 2, for juga dipakai di dalam for untuk menyusun angka di setiap baris. Sedangkan while dipakai kalau jumlah pengulangannya belum pasti, seperti pada soal nomor 1 yang baru berhenti setelah kasir memasukkan angka 0. Perintah break dan continue juga berguna untuk mengatur jalannya perulangan. Pada soal nomor 1, break dipakai untuk keluar dari perulangan saat jarak 0, dan continue dipakai untuk melewati jarak negatif. Pada soal nomor 4, continue dipakai untuk melewati mahasiswa yang IPK-nya kurang dari 3,00, dan break dipakai untuk menghentikan seleksi saat penerima beasiswa penuh sudah 3 orang. urutan perhitungan harus diperhatikan. Pada rumus tarif di soal nomor 1, kalau kurung pada (jarak - 5) tidak ditulis, hasilnya jadi salah karena Python mengerjakan perkalian lebih dulu.

5.2 Kesimpulan

Dari hasil praktikum, praktikan menyimpulkan bahwa perulangan di Python membuat program lebih singkat dan rapi karena perintah yang sama tidak perlu ditulis berulang-ulang, dengan catatan sebagai berikut :

a) Perulangan for cocok dipakai kalau jumlah pengulangannya sudah diketahui, sedangkan while dipakai kalau jumlah pengulangannya belum pasti.
b) Perintah break dipakai untuk menghentikan perulangan, sedangkan continue dipakai untuk melewati satu putaran dan langsung lanjut ke putaran berikutnya.
c) Syarat berhenti dan urutan perhitungan harus ditulis dengan teliti, misalnya memakai kurung pada rumus dan memastikan ada cara untuk keluar dari while, supaya hasil program benar dan tidak terjadi infinite loop.